

PE-EC801C — Error Correcting Codes

Semester VIII • 3L:0T:0P • 3 Credits

Module 4: Decoding of BCH and Reed-Solomon Codes

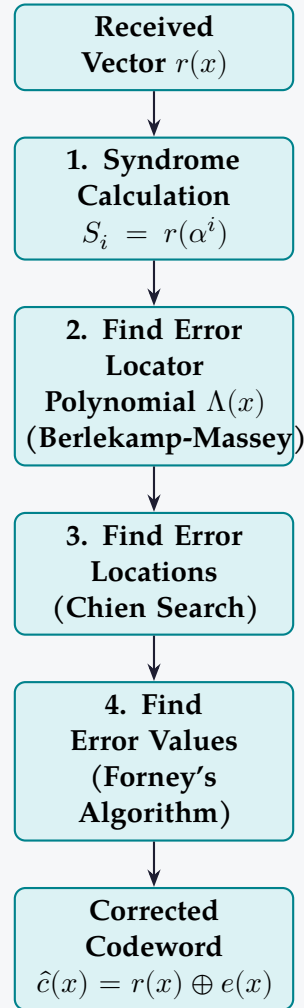
CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1. The BCH/RS Decoding Pipeline

Decoding algebraic codes (BCH and Reed-Solomon) is more complex than decoding linear block codes. Instead of simple table lookups, a multi-stage algebraic pipeline computes error locations and error values.

BCH and Reed-Solomon Decoding Flowchart



2. Syndrome Polynomial and the Key Equation

Let $c(x)$ be the transmitted codeword polynomial, $r(x)$ be the received polynomial, and $e(x) = \sum_{j \in \mathcal{E}} Y_j x^j$ be the error polynomial, where $\mathcal{E}$ is the set of error positions and Y_j are the error values ($Y_j \in \text{GF}(2)$ for binary BCH, and $Y_j \in \text{GF}(2^m)$ for RS codes).

2.1 Syndrome Calculations

The decoder calculates $2t$ syndromes:

$$S_i = r(\alpha^i) = c(\alpha^i) \oplus e(\alpha^i) = 0 \oplus e(\alpha^i) = \sum_{l=1}^{\nu} Y_l X_l^i, \quad i = 1, 2, \dots, 2t$$

where ν is the actual number of errors, $X_l = \alpha^{i_l}$ is the **error locator** indicating the position of the l -th error, and Y_l is the error value. We define the **Syndrome Polynomial**:

$$S(x) = \sum_{i=1}^{2t} S_i x^{i-1} = S_1 \oplus S_2 x \oplus S_3 x^2 \oplus \cdots \oplus S_{2t} x^{2t-1}$$

2.2 The Error Locator Polynomial

The **Error Locator Polynomial** $\Lambda(x)$ has roots that are the reciprocals of the error locators X_l :

Error Locator Polynomial

$$\Lambda(x) = \prod_{l=1}^{\nu} (1 - X_l x) = 1 + \Lambda_1 x + \Lambda_2 x^2 + \cdots + \Lambda_{\nu} x^{\nu}$$

The roots of $\Lambda(x)$ are $x_l = X_l^{-1}$. Finding these roots reveals the error positions.

2.3 The Key Equation

By defining the **Error Evaluator Polynomial** $\Omega(x)$ of degree less than ν , we establish the fundamental relationship of algebraic decoding:

The Key Equation of Decoding

$$\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$$

The task of the decoder is to solve this equation for $\Lambda(x)$ and $\Omega(x)$, given the known syndrome polynomial $S(x)$.

3. The Berlekamp-Massey Algorithm

The **Berlekamp-Massey (BM)** algorithm is an iterative design procedure that solves the key equation. It views the syndrome values as outputs of a Linear Feedback Shift Register (LFSR) and finds the minimum register length L and feedback connection polynomial $\Lambda^{(r)}(x)$ that can synthesize the sequence $S_1, S_2, \dots, S_{2t}$.

3.1 Iterative Steps of the BM Algorithm

We compute polynomials $\Lambda^{(r)}(x)$ for $r = 1, 2, \dots, 2t$:

Berlekamp-Massey Iteration

At step r , we compute the **discrepancy** d_r :

$$d_r = S_{r+1} \oplus \sum_{i=1}^L \Lambda_i^{(r)} S_{r+1-i}$$

1. If $d_r = 0$, the current LFSR synthesizes the next syndrome:

$$\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x), \quad L^{(r+1)} = L^{(r)}$$

2. If $d_r \neq 0$, the loop coefficients must be adjusted:

$$\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x) - d_r d_p^{-1} x^{r-p} \Lambda^{(p)}(x)$$

where p is a previous step that had a non-zero discrepancy d_p , chosen to minimize the new register length:

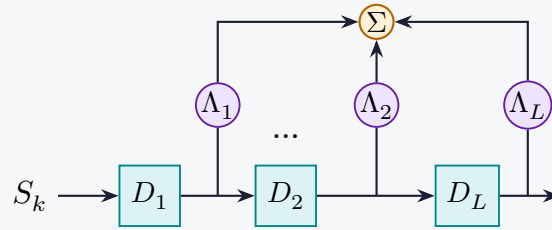
$$L^{(r+1)} = \max(L^{(r)}, r + 1 - L^{(p)})$$

After $2t$ iterations, the resulting polynomial $\Lambda^{(2t)}(x)$ is the desired error locator polynomial $\Lambda(x)$.

4. Massey's Shift Register Synthesis

James Massey showed that the algebraic problem of decoding BCH codes is equivalent to the system-theoretic problem of synthesizing a minimum-length linear feedback shift register that generates a given sequence.

Massey's LFSR Synthesis Model



The feedback connections $\Lambda_1, \dots, \Lambda_L$ represent the coefficients of the error locator polynomial $\Lambda(x) = 1 + \Lambda_1 x + \dots + \Lambda_L x^L$. The discrepancy d_r represents the difference between the actual syndrome S_{r+1} and the output predicted by the shift register.

5. A Fast Berlekamp-Massey Algorithm

To handle high-speed data transmission (e.g., optical networks and solid-state storage), the iterative Berlekamp-Massey algorithm can be reorganized into a parallel, systolic architecture. By utilizing division-free updates and maintaining auxiliary polynomials in parallel, modern decoders compute $\Lambda(x)$ in exactly $2t$ clock cycles without needing finite-field inversions (d_p^{-1}).

6. Chien Search and Forney's Algorithm

Once $\Lambda(x)$ and $\Omega(x)$ are computed, we must identify the error locations and calculate their values.

6.1 Chien Search

Chien search is a hardware-efficient method to find the roots of $\Lambda(x)$. Since the roots must lie in the finite field $\text{GF}(2^m)$, we test all possible elements α^{-i} ($i = 0, \dots, n-1$):

$$\Lambda(\alpha^{-i}) = 1 + \Lambda_1 \alpha^{-i} + \Lambda_2 \alpha^{-2i} + \dots + \Lambda_\nu \alpha^{-\nu i}$$

If $\Lambda(\alpha^{-i}) = 0$, then position i is identified as an error location. In hardware, the terms $\Lambda_j \alpha^{-ji}$ are updated each clock cycle using simple scaling registers, avoiding expensive exponentiations.

6.2 Forney's Algorithm

For non-binary codes (like Reed-Solomon), we must find the error values Y_l to correct the corrupted symbols. Forney's algorithm computes these directly:

Forney's Error Value Formula

The error value Y_l at position i_l (where $X_l = \alpha^{i_l}$) is:

$$Y_l = -\frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}$$

where $\Lambda'(x)$ is the formal derivative of the error locator polynomial:

$$\Lambda'(x) = \sum_{j=1}^{\nu} j \cdot \Lambda_j x^{j-1}$$

In binary fields, the addition is modulo-2, so the negative sign is ignored, and the derivative $\Lambda'(x)$ simplifies to the sum of its odd-indexed terms.

Quick Revision — Module 4

Decoding of BCH and RS Codes | PE-EC801C

Key Formulas and Metrics at a Glance

Syndrome elements	$S_i = \sum_{l=1}^{\nu} Y_l X_l^i$
Syndrome Polynomial	$S(x) = \sum_{i=1}^{2t} S_i x^{i-1}$
Error Locator Polynomial	$\Lambda(x) = \prod_{l=1}^{\nu} (1 - X_l x)$
Key Equation of Decoding	$\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$
Discrepancy equation	$d_r = S_{r+1} \oplus \sum_{i=1}^L \Lambda_i^{(r)} S_{r+1-i}$
Register length update	$L^{(r+1)} = \max(L^{(r)}, r+1 - L^{(r)})$
Chien search check	$\Lambda(\alpha^{-i}) = 0 \implies \text{error at index } i$
Forney's error value	$Y_l = -\frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}$
Formal derivative (binary)	$\Lambda'(x) = \Lambda_1 \oplus \Lambda_3 x^2 \oplus \Lambda_5 x^4 \oplus \dots$

One-Line Definitions

- **Syndrome Polynomial ($S(x)$):** A polynomial constructed from received syndromes representing the error pattern.
- **Error Locator ($\Lambda(x)$):** A polynomial whose roots indicate the positions where transmission errors occurred.
- **Error Evaluator ($\Omega(x)$):** A polynomial containing spectral details used to calculate error magnitudes.
- **Discrepancy (d_r):** The difference between the observed syndrome and the LFSR prediction in BM.
- **Berlekamp-Massey Algorithm:** An iterative, FSR-based procedure used to solve the key equation.
- **Chien Search:** A systematic search method evaluating $\Lambda(x)$ at all field elements to find roots.
- **Forney's Algorithm:** A low-complexity algebraic method used to calculate error values for non-binary codes.
- **Key Equation:** The core modular relationship linking $\Lambda(x)$, $S(x)$, and $\Omega(x)$ in algebraic decoding.

Frequently Asked / Viva Questions

Q1. What are the four main steps of the BCH/RS decoding pipeline?

First, compute the syndrome values from the received vector. Second, apply the Berlekamp-Massey algorithm to compute the error locator polynomial $\Lambda(x)$. Third, perform Chien search to find the roots of $\Lambda(x)$, locating the errors. Fourth, use Forney's algorithm to compute error values.

Q2. State the Key Equation of decoding and define its terms.

The Key Equation is $\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$, where $\Lambda(x)$ is the error locator polynomial, $S(x)$ is the syndrome polynomial, and $\Omega(x)$ is the error evaluator polynomial. It links the time and spectral representations of errors.

Q3. How does the Berlekamp-Massey algorithm model the decoding problem?

The BM algorithm models the syndrome sequence as outputs of a Linear Feedback Shift Register (LFSR). It iteratively constructs the shortest LFSR that generates the syndromes, which represents the minimum-degree error locator.

Q4. Explain the concept of 'discrepancy' in the BM algorithm.

Discrepancy (d_r) is the difference between the actual syndrome S_{r+1} and the value predicted by the feedback connections of the current iteration. If $d_r = 0$, the connections are correct; if $d_r \neq 0$, the connection polynomial must be adjusted.

Q5. What is Chien search, and how does it optimize hardware implementation?

Chien search finds the roots of $\Lambda(x)$ by evaluating it at all field elements α^{-i} . In hardware, terms are updated using multipliers that scale by constant values (powers of α), replacing complex exponentiations with simple shifts.

Q6. Explain Forney's algorithm and why it is useful.

Forney's algorithm calculates error values using $Y_i = -\Omega(X_i^{-1})/\Lambda'(X_i^{-1})$. It is highly useful because it allows direct computation of error values at known error locations without solving a large system of simultaneous equations.

Q7. Why does the formal derivative of a polynomial simplify in binary fields?

In binary fields, addition is modulo-2. Since $2 \equiv 0 \pmod{2}$, all terms with even powers in the derivative disappear (since their derivatives contain an even multiplier: $2 \cdot c \cdot x = 0$). Thus, only the odd-indexed terms remain.

Q8. Can the BM algorithm handle erasure decoding?

Yes. Erasures are errors with known locations but unknown values. The BM algorithm can be initialized with an erasure locator polynomial, reducing the number of syndrome iterations needed to solve for the remaining unknown errors.

Q9. What is the difference between Berlekamp-Massey and the Euclidean decoding algorithm?

BM is an iterative LFSR design process that runs in $2t$ steps. The Euclidean algorithm solves the key equation by finding the greatest common divisor of x^{2t} and $S(x)$ using polynomial division. Both produce identical results.

Frequently Asked / Viva Questions (contd.)

Q10. What happens during decoding if the number of channel errors exceeds t ?

If the number of errors exceeds t , the Berlekamp-Massey algorithm will still compute a polynomial, but it will have invalid roots, or the roots won't correspond to correct locations, resulting in a decoding failure or undetected error.

Comparison of Decoding Algorithms

Property	Berlekamp-Massey Algorithm	Euclidean Algorithm
Mathematical Basis	Linear feedback shift register synthesis	Polynomial GCD division
Execution Flow	Iterative (updates loop coefficients)	Recursive division steps
Hardware Suitability	Excellent (systolic array architectures)	Challenging (requires polynomial dividers)
Design Focus	Solving the LFSR feedback connections	Direct fraction reduction of $S(x)/x^{2t}$